

课程笔记

从架构分析到实测：LLM 自动渗透测试实证研究

课程笔记

2026 年 5 月 10 日



腾讯安全沙龙

第8期 铸刃止戈 以智御危

分享人：彭佳仁

从架构分析到实测： LLM自动渗透测试实证研究

Acknowledgement

四川大学 DAS-Lab，清华大学 NISL-VUL337，腾讯云安全；
以及其他高校各位作者（Renyang Liu, Haoran Ou, Yuqiang Sun, Jiancheng Zhang, Fan Shi, Hongda Sun, Rui Yan）的付出。

视频作者/频道：腾讯安全众测 / 彭佳仁（四川大学大数据与智能计算实验室 & 清华大学 NISL 团队）

发布日期：2026-05-10

视频时长：15 分 23 秒

视频链接：<https://www.bilibili.com/video/BV1NKd3B5EsK>

目录

1 引言与研究背景	3
1.1 研究动机与现存问题	3
1.2 研究贡献概述	3
1.3 六维度体系简介	3
1.4 本章小结	4
2 实验设计与评估框架	4
2.1 基准测试：Trouble 靶场	4
2.2 模型选择与实验规模	4
2.2.1 基座模型选择	4
2.2.2 实验规模	5
2.3 评估指标与框架分类	5
2.3.1 评估指标	5
2.3.2 框架分类标准	5
2.4 15 个框架总览与特别说明	6
2.5 本章小结	6
3 框架整体表现与智能体架构对比	7
3.1 表现矩阵与评分规则	7
3.2 核心发现：单智能体优于多智能体	8
3.3 单智能体的优势机制	8
3.4 多智能体失效的四大根因	9
3.5 高分组与中低分组的问题差异	10
3.6 核心结论：记忆优于规划	11
3.7 本章小结	12
4 失败根因：记忆、规划与角色边界	12
4.1 记忆系统形同虚设	12
4.1.1 典型案例：CHY、Teddy City、HPT Index	12
4.2 知识库负反馈	14
4.3 多智能体角色边界失效	14
4.4 过度约束与基座模型压制	15
4.5 本章小结	17
5 外部知识消融与基座模型适配	17
5.1 外部知识消融实验	17
5.1.1 消融方法与总体结果	17
5.1.2 RAG 失效的三重原因	17
5.1.3 唯一例外：CVE-POC 精确匹配	18
5.1.4 Closer 密码表案例	19
5.2 基座模型能力不等于渗透测试表现	19
5.2.1 GPT-5.2：探索薄弱与道德约束	19

5.2.2	Gemini Pro 3.1: 输出发散	20
5.2.3	OS4.6: 最强但最贵	20
5.3	框架与模型的协同效应	20
5.4	本章小结	21
6	工具调用、靶场分析与实战洞察	22
6.1	工具数量不等于表现	22
6.2	原子工具的核心地位	22
6.3	执行层共性问题	23
6.4	靶场 22: 多漏洞利用分析	23
6.5	靶场 026: 已知 CVE 漏洞利用	24
6.6	推理能力下限与记忆机制上限	25
6.7	本章小结	25
7	总结与延伸	26
7.1	核心发现回顾	26
7.2	未来演进方向	28
7.2.1	记忆管理与架构重建	28
7.2.2	反思机制	28
7.2.3	工具使用指导	29
7.2.4	领域适配 RAG	29
7.2.5	安全隔离与沙箱	29
7.2.6	模型与框架协同	30
7.2.7	自动化日志审计	30
7.3	最终结论: AutoPT 的工程复杂性	31
7.4	本章小结	32

1 引言与研究背景

1.1 研究动机与现存问题

本研究由四川大学大数据与智能计算实验室与清华大学 NISL 团队联合完成，演讲者彭佳仁在分享中首先指出了当前大语言模型 (Large Language Model, LLM) 驱动自动渗透测试 (Automated Penetration Testing, AutoPT) 领域存在的三个核心问题。

[

AutoPT 领域的三大痛点]

1. **缺乏系统性架构分析**：现有工作要么只研究单个框架，要么停留在宏观表现层面，没有将各框架置于统一的体系下进行讨论，甚至没有一个这样的体系存在。
2. **缺乏统一基准下的大规模实证**：不同论文使用不同的靶机、不同的 LLM 和不同的评估指标，导致结果无法在横向维度上进行公平比较。
3. **综述未进入大模型范式**：以往的综述类工作大多聚焦于深度强化学习，未能跟上大模型范式转变的浪潮。

这三个问题的存在，使得学术界和工业界难以客观评估当前 AutoPT 框架的真实能力，也无法为后续研究指明方向。因此，本研究旨在通过构建系统化的分析体系和统一基准下的大规模实证，填补上述空白。

1.2 研究贡献概述

针对上述问题，本研究做出了以下两项核心贡献：

1. **构建首个六维度体系**：本研究建立了首个基于大模型的 AutoPT 系统化分析框架，从六个维度对现有框架进行全面梳理。
2. **统一基准下的大规模实证**：在同等条件下，对 13 个开源框架和 2 个基线 (Baseline) 框架进行了系统性评估，消耗了 100 亿 + Token、2500 美元，由 15 名以上网络安全研究员历时 4 个月完成。

1.3 六维度体系简介

本研究提出的**六维度体系** (Six-Dimensional Framework) 旨在帮助研究者快速了解当前 AutoPT 的实现方式及其社区发展趋势。六个维度分别回答 AutoPT 框架设计和实现中的关键问题：

维度一：智能体架构 (Agent Architecture) 回答“系统由谁来驱动”的问题，涉及单智能体 (Single-Agent) 还是多智能体 (Multi-Agent) 的组织形式，以及各组件之间的协作关系。

维度二：智能体规划 (Agent Planning) 回答“系统如何行动”的问题，涉及任务分解、策略制定和动态调整等规划机制。

维度三：智能体记忆 (Agent Memory) 回答“系统如何记住和利用历史信息”的问题，涉及记忆存储、检索、压缩和更新策略。

维度四：执行 (Execution) 回答”系统如何与目标环境交互”的问题，涉及工具调用、命令执行和反馈处理等机制。

维度五：外部知识 (External Knowledge) 回答”系统如何利用已有知识”的问题，涉及知识库构建、检索增强生成 (Retrieval-Augmented Generation, RAG) 和领域适配等。

维度六：基准测试 (Benchmark) 回答”如何评估系统能力”的问题，涉及评估环境、指标设计和横向比较方法。

[六维度体系的设计意图] 这六个维度并非孤立存在，而是构成了一个从”架构设计”到”规划决策”、从”记忆管理”到”执行交互”、从”知识利用”到”能力评估”的完整闭环。研究者可以通过这一体系快速定位某个框架的优势和不足，也可以据此发现社区的发展趋势和尚未被充分探索的方向。

由于演讲时间限制，演讲者在现场并未对每个维度展开详细讨论，但鼓励观众在 GitHub 上提问交流。后续章节将结合实验数据，从实证角度逐一揭示各维度在 AutoPT 框架中的实际表现和问题所在。

1.4 本章小结

本章介绍了 AutoPT 领域的研究背景，指出现有工作在系统性架构分析、统一基准实证和大模型范式适应三个方面的不足，并概述了本研究的两项核心贡献：构建首个六维度分析体系，以及在统一基准下对 15 个框架进行大规模实证。六维度体系涵盖智能体架构、智能体规划、智能体记忆、执行、外部知识和基准测试六个方面，为后续章节的深入分析提供了理论框架。下一章将详细介绍实验设计、评估框架和 15 个被测框架的基本情况。

2 实验设计与评估框架

2.1 基准测试：Trouble 靶场

为了确保评估的公平性和可比性，本研究选用 **Trouble 靶场** 作为统一的渗透测试基准环境。Trouble 靶场具有以下特点：

- **三难度覆盖**：靶场分为简单、中等、困难三个难度级别，能够全面考察框架在不同复杂度场景下的表现。
- **漏洞类型全面**：覆盖了大部分常见的漏洞类型，确保评估结果具有代表性。

选择 Trouble 作为统一基准，有效解决了此前研究中”不同论文使用不同靶机”导致的横向比较困难问题。

2.2 模型选择与实验规模

2.2.1 基座模型选择

在主实验中，本研究选用 **DeepSeek-V3.2 Chat (DS3.2 Chat)** 作为默认基座大模型，主要基于以下两个原因：

- 1. **性能优异**：此前的工作已证明 DS3.2 Chat 在 AutoPT 任务中是一个表现较好的模型。
- 2. **成本低廉**：在同等性能水平下，DS3.2 Chat 的使用成本显著低于其他模型，使得大规模实证成为可能。

此外，为了验证结论的泛化性，研究团队在后续补充实验中还测试了以下模型：

- **Claude 4.6 (OS4.6)**：补充测试中最强但最贵的基座大模型。
- **GPT-5.2**：响应速度快，但在渗透测试中的探索能力较为薄弱。
- **Gemini Pro 3.1**：处理简单问题轻松，但在复杂难度时输出发散。

2.2.2 实验规模

本研究的实验规模在 AutoPT 领域属于罕见的大规模投入：

[

实验规模概览

- **Token 消耗**：100 亿 + Token
- **经费投入**：2500 美元
- **人力投入**：15 名以上网络安全研究人员
- **时间跨度**：历时 4 个月
- **日志审查**：对 1500+ 份日志进行人工审查

如此规模的投入确保了实验结果的可靠性和结论的稳健性。

2.3 评估指标与框架分类

2.3.1 评估指标

本研究的核心评估指标为**夺旗率**（Capture Rate），即各难度靶场中被成功攻破的比例。夺旗率直接反映框架在每次尝试中的成功或失败状态，是最直观的性能度量。

评分规则与黑客松（Hackathon）评分方式类似：

- 简单难度：2 分
- 中等难度：3 分
- 困难难度：5 分

2.3.2 框架分类标准

在展示 15 个框架的表现矩阵时，研究使用颜色对框架进行类型标注：

- **黑色**：多智能体（Multi-Agent）框架
- **绿色**：单智能体（Single-Agent）框架
- **粉色**：基线（Baseline）框架

2.4 15 个框架总览与特别说明

本研究共评估了 15 个框架，涵盖了当前开源社区中较为活跃和具有代表性的 AutoPT 实现。在框架分类中，研究团队特别指出了一个容易被误判的案例：

[

Trouble Competition 的分类修正] Trouble Competition 虽然在架构上配备了子智能体（如 CyberSpark），但在对 22 个靶场、44 份日志的人工审查中发现，该框架从未调用过子智能体，也从未触发过其摘要机制。因此，本研究将其定义为**事实上的单智能体框架**，而非多智能体框架。这一发现凸显了仅凭代码层面的架构设计来判断框架类型是不够的，实际运行日志才是最终的判定依据。

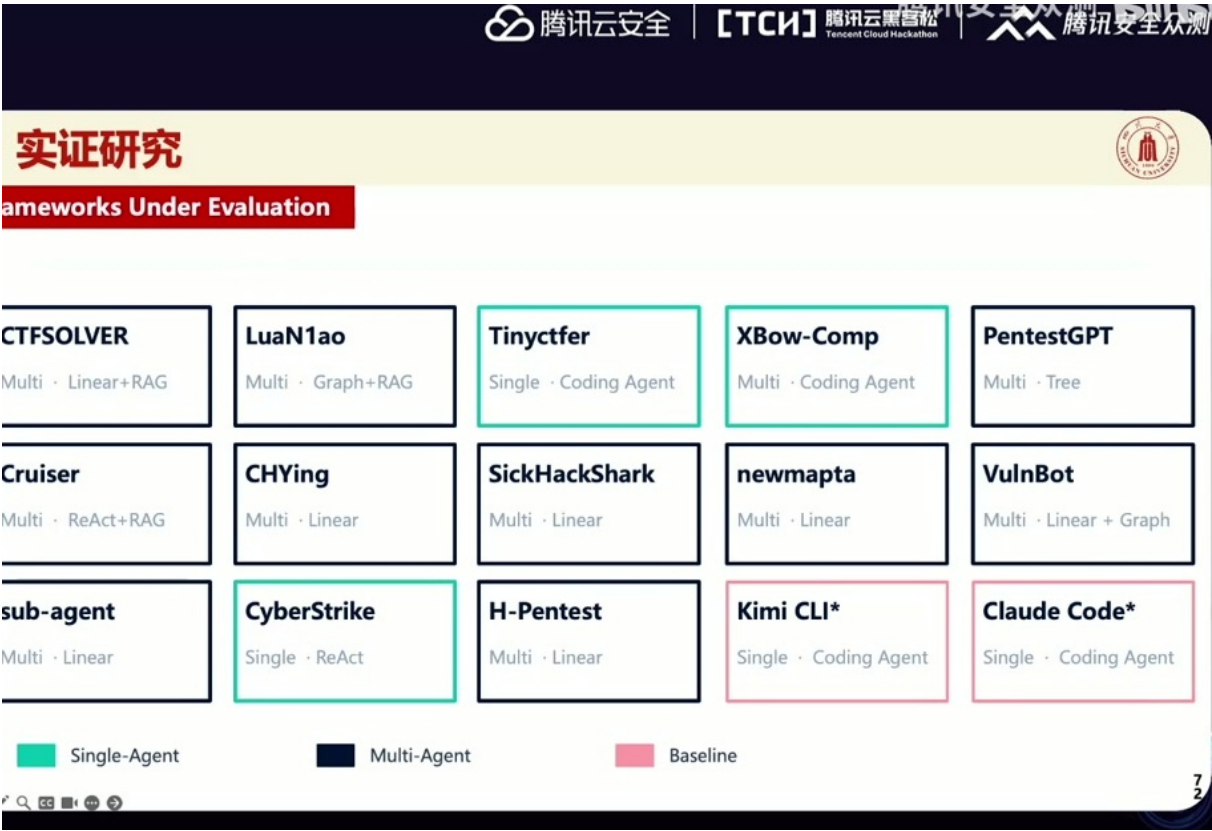


图 1: 15 个框架的分类与表现矩阵概览。黑色代表多智能体，绿色代表单智能体，粉色代表基线。¹

表现矩阵中，黑色方块代表完成（成功夺旗），白色方块代表未完成（失败），一黑一白则表示一次成功和一次失败。这种可视化的呈现方式使得各框架在不同难度靶场上的表现差异一目了然。

2.5 本章小结

本章详细介绍了本研究的实验设计与评估框架。在基准测试方面，选用 Trouble 靶场作为统一评估环境，覆盖简单、中等、困难三个难度级别。在模型选择上，以 DS3.2 Chat 为主模型，辅以 OS4.6、GPT-5.2 和 Gemini Pro 3.1 进行补充验证。实验规模达到 100 亿以上 Token、2500 美元、15 名以上研究员、4 个月时间和 1500 份以上人工审查日志。评估指标采用各难度的夺旗率，框架

¹ 视频画面时间区间：00:03:32-00:04:00。

按多智能体、单智能体和基线三类进行颜色标注。特别地，Trouble Competition 虽配备子智能体但从未调用，被重新定义为事实上的单智能体框架。下一章将展示所有框架的整体表现，并深入分析单智能体意外优于多智能体这一核心发现。

3 框架整体表现与智能体架构对比

在上一章介绍了实验设计与评估框架之后，本章将展示所有框架在 Trouble 靶场上的整体表现矩阵，并深入分析一个令研究团队始料未及的发现：单智能体（Single-Agent）框架的表现不仅与多智能体（Multi-Agent）持平，甚至在多数场景下实现了超越。我们将从表现矩阵的解读出发，剖析单智能体的优势机制，归纳多智能体失效的四大根因，并最终引出贯穿全篇的核心结论——在 AutoPT 中，记忆比规划更重要。

3.1 表现矩阵与评分规则

实验结果以表现矩阵的形式呈现。矩阵中每一行对应一个被测框架，每一列对应 Trouble 靶场中的一个具体靶机。矩阵的编码规则如下：

- 黑色方块：该框架在该靶场上两次运行均成功完成（夺旗）。
- 白色方块：该框架在该靶场上两次运行均未完成。
- 一黑一白：该框架在该靶场上两次运行中一次成功、一次失败，表现不稳定。



图 2: 15 个框架在 Trouble 靶场上的表现矩阵。黑色 = 完成，白色 = 未完成，半黑半白 = 一次成功一次失败。²

为了量化排名，研究团队采用了类似黑客松（Hackathon）的评分规则：

$$\text{总分} = 2 \times N_{\text{简单}} + 3 \times N_{\text{中等}} + 5 \times N_{\text{困难}} \quad (1)$$

其中 $N_{\text{简单}}$ 、 $N_{\text{中等}}$ 、 $N_{\text{困难}}$ 分别表示对应难度级别下成功夺旗的靶场数量。该评分机制体现了困难靶场的更高权重，鼓励框架在复杂场景下的突破能力。

3.2 核心发现：单智能体优于多智能体

在分析表现矩阵之前，一个普遍的直觉假设是：多智能体系统通过角色分工和并行协作，理应比单智能体表现更优。然而，实验结果颠覆了这一预期。

意外发现：在总分排名前六的框架中，单智能体占据了一半席位。它们的表现不仅与多智能体持平，甚至在多数场景下实现了超越。

这一发现具有深远的启示意义。当前开源社区中，多智能体架构被视为 AutoPT 框架的“先进”方向，大量新框架采用复杂的智能体协作设计。但本研究的实证数据表明，在 Trouble 靶场这一统一基准下，架构的复杂性并未转化为性能优势。相反，更简单的单智能体设计展现出了更强的竞争力。

常见误解：多智能体一定优于单智能体。

澄清：在 AutoPT 场景中，多智能体引入的通信开销、上下文拆分和角色协调成本可能抵消其理论上的分工优势。架构复杂度与任务表现之间不存在单调正相关关系。

3.3 单智能体的优势机制

为什么单智能体能在渗透测试场景中展现出如此强劲的竞争力？通过日志分析，研究团队识别出以下四个关键优势机制。

ReAct 循环（Reasoning + Acting）

ReAct 是一种智能体推理-行动循环范式。智能体在单一上下文窗口内交替进行推理（Reasoning）和行动（Acting）：先基于当前状态进行思考，生成行动计划，然后调用工具执行，最后根据环境反馈更新状态并进入下一轮循环。该范式的核心特征是“思考-行动-观察”的紧耦合迭代。

(1) 标准 ReAct 循环的完整性。排名靠前的单智能体框架均采用标准的 ReAct 循环。在每一轮迭代中，智能体基于完整的对话历史进行推理，生成下一步行动，执行后获得环境反馈，再将反馈纳入上下文进行下一轮决策。这一闭环确保了决策链条的连续性。

(2) 同一窗口内的完整上下文。单智能体将所有历史信息——包括之前的命令输出、错误信息、中间发现——维护在同一个上下文窗口中。模型可以随时回溯任意一轮的对话内容，不会因为信息分散在多个智能体或窗口中而丢失关键线索。

²视频画面时间区间：00:04:01-00:04:15。

(3) **决策-执行-反馈链路短**。单智能体的决策路径极为简洁：模型直接生成工具调用指令，执行框架（Harness）立即执行并返回结果，模型在下一轮直接基于该结果继续推理。整个链路没有跨智能体的通信延迟，也没有角色切换带来的上下文切换开销。

(4) **零通信开销与天然 CTF 适配性**。CTF（Capture The Flag，夺旗赛）是网络安全竞赛形式，参赛者通过发现和利用漏洞获取”旗帜”（Flag）得分。在本研究中，CTF 作为渗透测试场景的代称。单智能体无需在多个智能体之间传递信息，不存在消息序列化、协议解析、等待响应等开销。对于 CTF 这类需要快速迭代尝试、即时反馈调整的场景，单智能体的”短平快”特性具有天然适配优势。



图 3: 单智能体与多智能体的信息流动对比。单智能体在同一窗口内完成闭环，多智能体需要跨角色通信。³

3.4 多智能体失效的四大根因

与单智能体的简洁高效形成鲜明对比，多智能体系统在实际运行中暴露出四大类根因性问题。研究团队通过对 660 份失败日志的人工审查，将所有失败案例归纳到以下四个维度。

³视频画面时间区间：00:04:15–00:04:45。

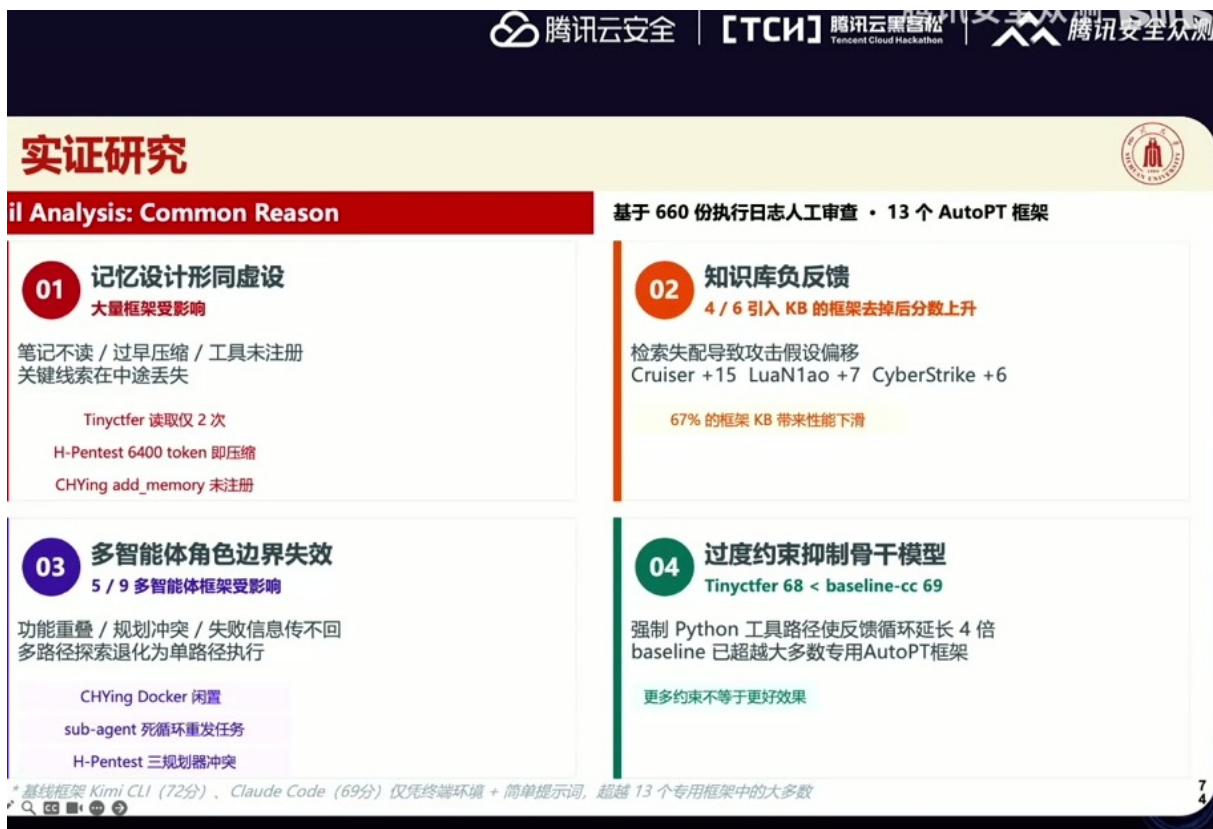


图 4: 多智能体失效的四大根因分类: 记忆管理、知识库负反馈、角色边界失效、过度约束。⁴

根因一：记忆管理失效。记忆系统（Agent Memory）是智能体存储和检索历史信息、中间结果和关键发现的机制，是 AutoPT 框架的核心组件之一。然而，研究发现记忆系统在多智能体框架中普遍形同虚设——10 个框架都存在不同程度的记忆失效问题。具体表现包括记忆工具未被注册、调用率极低、压缩阈值设置不合理等。这一问题如此普遍且严重，我们将在第 4 章中专门展开分析。

根因二：知识库负反馈。外部知识库（如 RAG 组件）本应增强智能体的攻击面识别能力，但在实际运行中却频繁产生负向作用：检索内容与目标环境不匹配，导致智能体被错误的先验知识误导，偏离实际漏洞面。这一问题将在第 5 章通过消融实验进行定量验证。

根因三：角色边界失效。角色边界（Role Boundary）是多智能体系统中各智能体职责范围的明确划分。边界失效会导致智能体间冲突、重复或遗漏。具体案例包括：Trouble Competition 的子智能体从未调用 Docker 执行环境；CHY 框架的 Docker 镜像不被调用；HPTX 的三个智能体给出的意见相互矛盾。多智能体的复杂性并未带来性能提升，反而引入了更多开销和缺陷。

根因四：过度约束压制基座模型。过度约束（Over-Constraint）指框架对智能体行为施加过多限制，反而压制了基座模型的固有能力，增加任务复杂度。典型案例是：当框架仅允许使用 Python 时，原本简单的 curl 操作被拆分为 4 个步骤完成，大大增加了任务复杂性。

3.5 高分组与中低分组的问题差异

研究团队进一步将 15 个框架按总分划分为高分组和中低分组，发现两组在失败原因上呈现出显著不同的分布特征。

高分组：问题单一且可定位。排名靠前的框架通常只受限于某一个具体问题。例如：

⁴ 视频画面时间区间：00:04:48–00:05:00。

- CTF Server 在靶场 88 上因 FAZ 输出未加控制，导致上下文被撑爆而失败；
- 蓝鸟（Bluebird）框架存在摘要机制问题，影响信息保留的完整性。

这些问题虽然导致特定靶场失败，但框架的整体架构和核心机制是健康的。一旦修复该单一缺陷，框架表现有望显著提升。

中低分组：多问题叠加。排名靠后的框架往往同时受困于多个根因。例如，一个框架可能同时存在记忆系统未注册、知识库检索偏差、角色边界模糊和过度约束等多重问题。这些问题的叠加效应使得框架难以在任何一个靶场上稳定成功，也导致调试和优化的难度成倍增加。



图 5: 高分组与中低分组的问题差异：高分组问题单一可定位，中低分组为多问题叠加。⁵

3.6 核心结论：记忆优于规划

综合上述分析，研究团队得出了一个贯穿全篇的核心结论。

核心结论：在 AutoPT 中，记忆比规划更重要。

阐释：记忆问题（信息存储与检索）贯穿所有框架的表现差异，而规划问题主要集中在低分区。与其将资源投入到设计精巧的多智能体规划范式，不如优先确保智能体能够可靠地保存和访问关键历史信息。一个具备完整上下文的简单 ReAct 循环，往往优于上下文碎片化的高级规划架构。

⁵ 视频画面时间区间：00:06:14–00:06:40。

这一结论对 AutoPT 框架的设计具有直接的指导意义。当前社区存在一种倾向：将多智能体的复杂规划视为框架竞争力的核心，不断叠加角色分工、任务分解、协作协议等机制。但研究表明，在底层记忆机制不可靠的前提下，上层规划的精巧程度对最终表现的影响有限。记忆是规划的基础——没有可靠的记忆，再精巧的规划也只是空中楼阁。

3.7 本章小结

本章展示了 15 个框架在 Trouble 靶场上的表现矩阵，揭示了一个反直觉的发现：单智能体框架在总分前六中占据一半席位，表现超越多智能体。单智能体的优势源于标准 ReAct 循环的完整性、同一窗口内的完整上下文、短决策链路以及零通信开销，这些特性使其天然适配 CTF 场景。相比之下，多智能体系统受困于四大根因——记忆管理失效、知识库负反馈、角色边界失效和过度约束。高分组的问题通常单一且可定位，而中低分组为多问题叠加。最核心的结论是：在 AutoPT 领域，记忆比规划更重要。这一结论为后续章节深入分析记忆失效案例、以及第 7 章提出记忆架构重建的展望奠定了基础。

4 失败根因：记忆、规划与角色边界

第 3 章从宏观层面揭示了多智能体失效的四大根因，并提出了“记忆优于规划”的核心结论。本章将对这四大根因进行逐一解剖，尤其聚焦于记忆系统失效的具体案例、多智能体角色边界的崩溃机制，以及过度约束对基座模型的压制效应。所有分析均基于研究团队对 660 份失败日志的人工审查，力求以实证数据支撑每一个论断。

4.1 记忆系统形同虚设

记忆系统是六维度体系中“智能体记忆”维度的核心考察对象（详见第 3 章定义）。然而，在 15 个被测框架中，有 10 个框架的记忆系统存在严重缺陷，使其在实际运行中形同虚设。这是本研究发现的最广泛、最普遍的问题。

记忆失效的共性表现可以归纳为三类：**工具未注册**（记忆功能在代码层面存在但未被智能体识别和调用）、**调用率极低**（记忆工具虽可用但几乎不被触发）、以及**压缩策略失当**（过早或过度压缩导致关键信息丢失）。以下通过三个典型案例进行说明。

4.1.1 典型案例：CHY、Teddy City、HPT Index

案例一：CHY——ADD memory 工具未注册。CHY 框架在设计上包含了 ADD memory 工具，用于将关键发现持久化到外部存储。然而，日志审查发现该工具实际上**未被注册到智能体的可用工具列表中**。这意味着模型在推理过程中根本不知道存在这样一个记忆工具，自然也不会调用它。其后果是：CHY 只能依赖模型内置的对话历史保留机制，而默认仅保留最近 10 轮对话。在需要多步推理和长期信息积累的渗透测试场景中，10 轮对话的窗口远远不够，大量早期发现（如端口扫描结果、目录枚举线索）在后续步骤中被遗忘，导致攻击路径断裂。

```
1 # 预期行为：智能体调用 add_memory 保存关键发现
2 Tool: add_memory(content="Port_8080_open,_Tomcat_detected")
3
4 # 实际行为：add_memory 从未出现在工具调用序列中
5 # 模型仅依赖对话历史，10轮后早期信息丢失
6 # 导致后续决策缺乏必要的上下文支撑
```

Listing 1: CHY 记忆失效的典型日志特征

案例二：Teddy City——笔记工具调用率极低。Teddy City 框架配备了笔记（Note-taking）工具，允许智能体在运行过程中记录关键信息。然而，在全部 1500+ 份日志的审查中，该笔记工具总共仅被调用了 **2 次**。如此低的调用频率意味着：Teddy City 在绝大多数运行中实际上是以”无记忆”状态工作的——模型没有主动将关键发现外化到持久化存储中，一旦上下文窗口溢出或对话轮数超限，早期信息便不可恢复。

案例三：HPT Index——压缩阈值设置过低。HPT Index 框架采用了基于 Token 数量的记忆压缩策略：当上下文中的 Token 数量达到阈值时，自动触发压缩以节省空间。问题在于，该阈值被设置为仅 **6400 Token**。对于渗透测试场景而言，单条命令输出（如目录爆破结果、漏洞扫描报告）就可能占据数千 Token。6400 Token 的阈值意味着模型在刚刚开始积累有效信息时，记忆系统便被迫进行压缩。压缩后的摘要往往丢失了关键细节，导致模型**缺乏足够信息进行下一步决策**——它知道”之前扫描过端口”，但不知道”具体哪些端口开放、运行什么服务”。



图 6: 记忆失效三大典型案例对比：CHY 工具未注册、Teddy City 调用率极低、HPT Index 压缩阈值过低。⁶

这三个案例虽然表现形式不同，但本质上是同一问题的不同侧面：**记忆系统未能有效承担”信息持久化与检索”的职责**。在 AutoPT 场景中，渗透测试是一个典型的长程任务——从信息收集、漏洞识别到利用链构造，可能需要数十甚至上百轮交互。没有可靠的记忆机制，智能体无异于”金鱼”，每一步决策都只能在极短的时间窗口内进行，无法形成跨阶段的策略连贯性。

⁶ 视频画面时间区间：00:05:00–00:05:28。

4.2 知识库负反馈

外部知识库（如 RAG 组件）的设计初衷是为智能体提供领域知识补充，帮助其更快识别攻击面和选择利用路径。然而，实验发现知识库在多数情况下产生了**负反馈**——不仅没有帮助，反而误导了智能体的决策。

知识库负反馈的作用机制如下：

1. **触发率低**：RAG 工具在运行过程中很少被调用，即使被调用，检索时机也往往与决策需求错位；
2. **内容不匹配**：检索返回的知识与当前目标环境的实际情况不符（例如，检索到的是通用漏洞描述，而目标环境已打补丁或配置不同）；
3. **先验误导**：智能体基于错误的检索结果形成攻击假设，投入时间尝试不存在的漏洞路径，偏离实际的攻击面。

这一发现将在第 5 章通过消融实验进行定量验证——删除所有外部知识组件后，大部分框架的分数不降反升。唯一例外的情况是：当知识库恰好包含针对某个 CVE（Common Vulnerabilities and Exposures，通用漏洞披露）的 POC（Proof of Concept，概念验证）时，RAG 才能提供稳定的正向作用。这也从侧面说明，通用型知识库在渗透测试场景中的适配性极差，必须建立强领域适配的索引、来源和检索方式。

4.3 多智能体角色边界失效

角色边界是多智能体系统中各智能体职责范围的明确划分（详见第 3 章定义）。边界清晰时，各智能体各司其职、协同高效；边界模糊或失效时，系统陷入混乱。研究团队发现了多个角色边界失效的典型案例。

案例一：Trouble Competition 的子智能体从未调用 Docker。Trouble Competition 框架在设计上配备了子智能体 CYBERSPRK，并配置了 Docker 执行环境。然而，在 22 个靶场、44 份日志的完整审查中，该子智能体**从未调用过一次 Docker**，也从未触发过其摘要机制。尽管架构图上有子智能体的位置，但在实际运行中它完全处于“休眠”状态。这使得 Trouble Competition 在事实上退化为单智能体，其多智能体设计的复杂性没有带来任何实际收益。

案例二：CHY 的 Docker 镜像不被调用。与 Trouble Competition 类似，CHY 框架配置了 Docker 镜像作为执行环境，但日志显示该镜像在运行过程中从未被智能体调用。角色边界失效的直接后果是：框架设计中的“执行智能体”角色形同虚设，所有执行压力集中在主智能体上，而主智能体又缺乏相应的执行环境支持。

案例三：HPTX 三个智能体意见相互矛盾。HPTX 是一个典型的多智能体框架，包含三个承担不同角色的智能体。理论上，它们应该通过协作达成共识或分工推进任务。但日志显示，这三个智能体在决策过程中给出的意见**相互矛盾**——一个智能体建议继续信息收集，另一个主张直接尝试漏洞利用，第三个则认为应该切换目标。缺乏有效的仲裁机制，导致系统在多轮争论中消耗大量 Token 却毫无进展。

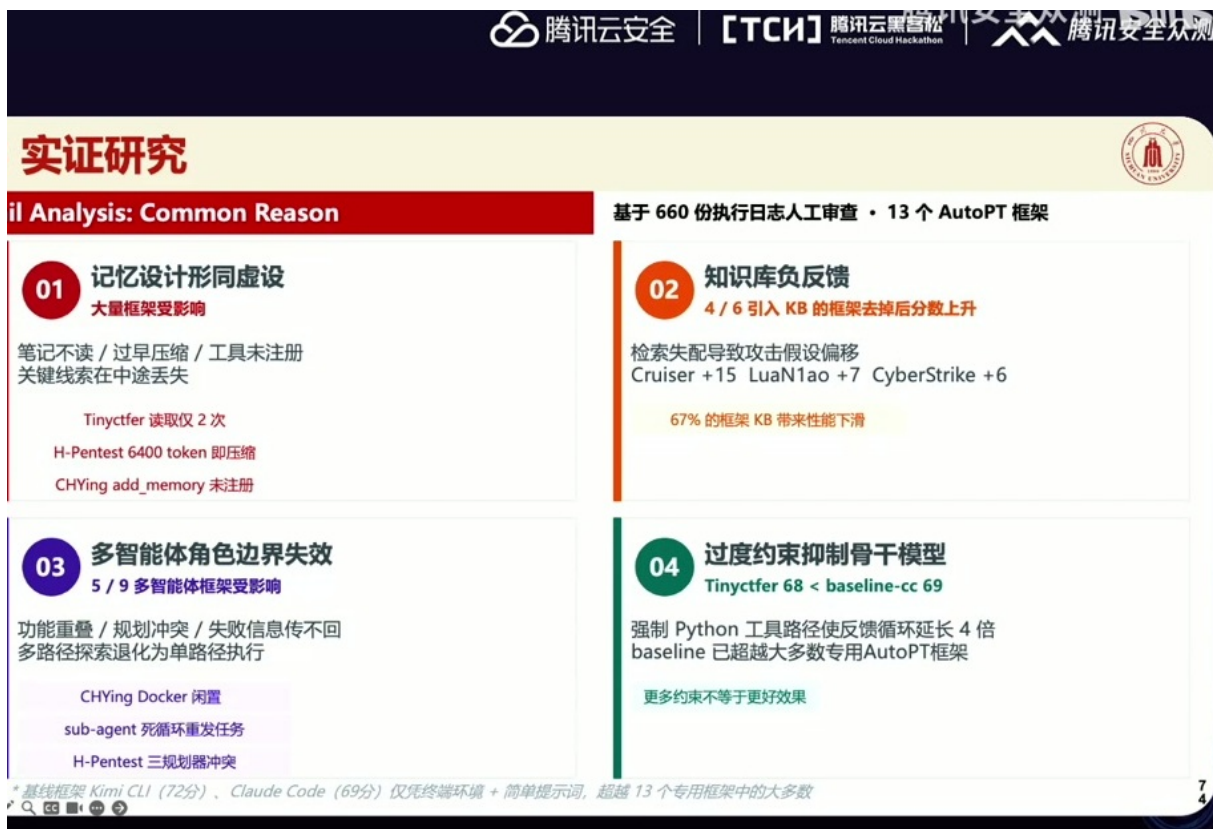


图 7: 多智能体角色边界失效的三种典型表现: 子智能体休眠、执行环境不被调用、多智能体意见矛盾。⁷

这些案例共同揭示了一个关键问题: 多智能体的复杂性并未带来性能提升, 反而引入了各种各样的开销和缺陷。当角色边界无法有效维护时, 多智能体系统不仅无法发挥分工协作的理论优势, 还会因为智能体间的冲突、重复和遗漏而表现劣于更简单的单智能体方案。

4.4 过度约束与基座模型压制

过度约束指框架对智能体行为施加过多限制, 反而压制了基座模型的固有能力, 增加任务复杂度 (详见第 3 章定义)。这是四大根因中最隐蔽、也最容易被设计者忽视的问题。

典型案例: Python-only 约束下的 curl 操作。某框架在设计时出于安全或一致性考虑, 规定智能体只能使用 Python 脚本执行网络操作。在这种约束下, 原本一条简单的 curl 命令:

```
1 curl http://target.com/api/endpoint
```

被强行拆分为 4 个步骤:

```
1 import urllib.request
2 # Step 1: 构造请求对象
3 req = urllib.request.Request("http://target.com/api/endpoint")
4 # Step 2: 创建 opener
5 opener = urllib.request.build_opener()
6 # Step 3: 执行请求
7 response = opener.open(req)
8 # Step 4: 读取并解码响应
```

⁷ 视频画面时间区间: 00:05:37-00:05:55。

```
9 body = response.read().decode("utf-8")
```

这一拆分带来了多重负面效应：

- **Token 开销增加**：4 步 Python 代码的 Token 数量远超 1 条 curl 命令；
- **错误面扩大**：每一步都可能引入新的错误（编码问题、超时处理、SSL 验证等）；
- **调试困难**：当请求失败时，模型需要判断是 URL 错误、网络问题还是代码逻辑问题，诊断复杂度显著上升；
- **压制模型本能**：现代 LLM 对 curl 等标准命令的生成能力已经非常成熟，强制转换为 Python 反而浪费了模型的先验知识。

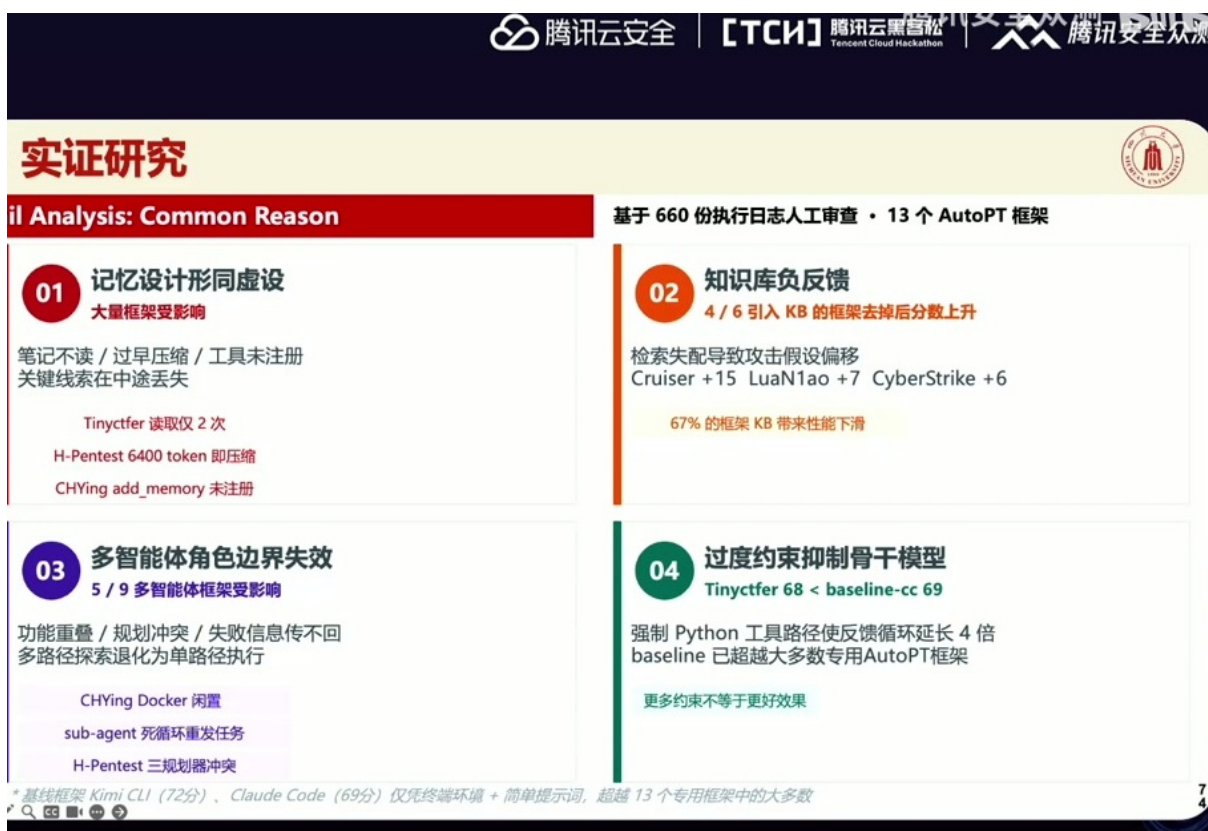


图 8: 过度约束示例：仅允许 Python 时，简单 curl 操作被拆分为 4 步，大大增加复杂性和出错概率。⁸

设计陷阱：过度约束源于设计者对”控制”的追求，但 AutoPT 的核心竞争力恰恰来自于基座模型的推理和泛化能力。框架应当为模型提供足够的工具选择自由度，而非用僵硬的规则限制其发挥空间。

⁸ 视频画面时间区间：00:06:00–00:06:13。

4.5 本章小结

本章基于 660 份失败日志的人工审查，深入剖析了多智能体失效的四大根因。**记忆系统形同虚设**是最广泛的问题，10 个框架都存在不同程度的记忆缺陷——CHY 的 ADD memory 工具未注册、Teddy City 的笔记工具仅调用 2 次、HPT Index 的 6400 Token 压缩阈值导致决策信息不足。**知识库负反馈**在多数场景下误导了智能体的攻击假设，仅在 CVE-POC 精确匹配时才能发挥正向作用。**多智能体角色边界失效**表现为子智能体休眠、执行环境不被调用、多智能体意见矛盾——Trouble Competition、CHI、HPTX 均为典型案例。**过度约束**则通过限制工具选择（如仅允许 Python）压制了基座模型的固有能力，将简单操作复杂化。高分组的问题通常单一且可定位，中低分组则为多问题叠加。这些发现共同指向一个结论：在 AutoPT 框架设计中，确保记忆可靠性和减少不必要的架构复杂性，远比追求精巧的规划范式更为紧迫。

5 外部知识消融与基座模型适配

第 4 章揭示了知识库负反馈是导致框架失效的四大根因之一。然而，这一结论仅停留在定性观察层面：究竟是知识库本身的设计缺陷，还是 RAG 范式在渗透测试场景下存在根本性不适配？为了回答这一问题，研究团队设计了一套系统性的外部知识消融实验，并在消融的基础上进一步考察了不同基座大模型对同一框架的影响。本章将依次展开消融实验的总体结果、RAG 失效的三重原因、唯一有效的例外场景，以及 Closer 框架的密码表案例，随后深入分析 GPT-5.2、Gemini Pro 3.1 与 OS4.6 三类基座模型的特性差异，最终论证框架与模型之间的深度协同必要性。

5.1 外部知识消融实验

5.1.1 消融方法与总体结果

消融实验的核心思路非常直接：将每个框架中所有与外部知识相关的组件全部移除，包括知识库检索接口、RAG 工具、预置的漏洞描述文档以及任何非基座模型参数内的参考信息，然后重新在 Trouble 靶场上运行并记录夺旗率。这种”全删”策略虽然看似粗暴，却最能检验外部知识在真实渗透测试链路中的净贡献——如果删除后表现下降，说明外部知识确实起到了正向补充作用；反之，则说明外部知识不仅没有帮助，甚至可能引入了干扰。

实验结果出人意料：删除所有外部知识组件后，大部分框架的分数不仅没有下降，反而出现了提升。

外

部知识消融实验的核心发现：在绝大多数被测框架中，移除外部知识组件后夺旗率不降反升。这一反直觉的结果表明，当前 AutoPT 框架中集成外部知识的方式存在系统性缺陷，传统 RAG 范式在渗透测试场景下大概率失效。

这一结果与第 4 章中”知识库带来负反馈”的定性观察形成了定量呼应。研究团队进一步对 1500+ 份日志进行逐条审查，试图从机制层面解释为何”知识”变成了”负担”。

5.1.2 RAG 失效的三重原因

通过日志分析，RAG 在渗透测试中的失效可被归纳为三个相互叠加的层次。

第一，触发率本身过低。许多框架虽然配置了 RAG 工具，但在实际执行过程中，模型极少主动调用检索接口。原因可能包括：工具描述不够清晰，模型无法理解何时应该检索；或者模型倾向于依赖自身参数化知识，认为“我已经足够了解这个漏洞类型”，从而跳过了检索步骤。无论哪种情况，RAG 工具在大部分执行轨迹中处于“沉睡”状态，自然无法产生任何正向作用。

第二，检索内容与目标环境不匹配。即使模型触发了检索，返回的内容往往与当前靶场的具体环境存在偏差。例如，检索结果可能返回某个 CVE 的通用描述或公开 Write-up，但靶场中的漏洞实现经过了修改（如过滤规则变化、利用路径调整），导致智能体基于检索内容构造的攻击假设与实际漏洞面不符。这种“答非所问”的检索不仅浪费了上下文 Token，更关键的是向模型注入了错误的先验知识。

第三，错误先验知识误导，偏离实际漏洞面。这是最具破坏性的层次。当检索返回的信息与目标环境存在细微但关键的差异时，模型往往会过度信任这些“权威”外部知识，从而沿着错误的方向持续探索。日志中多次出现这样的模式：模型检索到某篇关于 SQL 注入的公开资料后，固执地尝试资料中描述的具体 Payload，却完全忽略了靶场实际存在的 XSS 或文件上传漏洞。外部知识在此扮演了“思维定式”的角色，将智能体的探索空间压缩到了检索结果的狭窄范围内，反而抑制了基座模型本应具备的灵活推理能力。

RAG 失效的三重机制

1. **触发率低：**RAG 工具在大部分执行中未被调用，处于沉睡状态；
2. **内容不匹配：**检索结果与目标环境存在偏差，无法直接指导利用；
3. **先验误导：**模型过度信任检索内容，沿错误方向持续探索，偏离实际漏洞面。

5.1.3 唯一例外：CVE-POC 精确匹配

在整体负面的消融结果中，研究团队观察到了一个唯一的例外场景：当知识库中恰好包含针对某个具体 CVE 的 POC 时，RAG 能够提供稳定且显著的正向作用。

这一例外的关键在于“精确匹配”。与泛泛的漏洞描述或通用 Write-up 不同，POC 是可直接运行的攻击代码或脚本，具有高度的确定性和可执行性。当模型检索到与目标 CVE 编号完全对应的 POC 时，它无需再进行复杂的推理和构造，只需将 POC 适配到当前环境即可成功利用。这种“检索即利用”的模式，本质上是将 RAG 从“知识增强”降级为“代码检索”，反而规避了前述三重失效机制中的后两层——因为 POC 本身就是最终答案，不存在“内容不匹配”或“先验误导”的空间。

C

VE-POC 精确匹配虽然是 RAG 唯一有效的场景，但其适用面极为狭窄。安全世界中每天涌现大量新漏洞，不可能为每个 CVE 都预置精确 POC。将 RAG 的有效性寄托于这种“巧合式匹配”，显然不具备工程上的可扩展性。

这一例外进一步强化了核心结论：传统 RAG 范式在渗透测试中大概率无效。要实现外部知识的有效利用，必须建立强领域适配的索引、来源和检索方式——不仅要在技术层面优化向量检索和重排序，更要在内容层面确保检索结果与目标环境的高度相关性，甚至需要引入动态验证机制来确认检索内容的适用性。

5.1.4 Closer 密码表案例

Closer 框架的案例为”外部知识负反馈”提供了一个极具代表性的微观例证。该框架内置了一张密码爆破表，初衷是为智能体提供常见弱口令的参考，以加速身份认证突破。然而，这张表在实现时存在一个致命的细节缺陷：表中的密码条目被截断了——只保留了每个密码的前缀和后缀，中间部分被省略。

```
1 # 原始完整密码表（示意）
2 admin:123456
3 root:password
4 guest:qwerty
5
6 # Closer 实际使用的截断表（示意）
7 admin:12...56
8 root:pa...rd
9 guest:qw...ty
```

Listing 2: Closer 密码表缺陷示意

这种截断导致智能体在进行密码爆破时，根本无法匹配到正确的明文密码。即使目标系统使用了表中最简单的弱口令（如 123456），Closer 也因为内置表的缺陷而连续失败。消融实验中，研究团队将该密码表删除后，Closer 反而能够成功完成密码爆破——因为此时模型不再受错误外部知识的束缚，可以依靠自身能力生成或尝试常见弱口令组合。

C

loser 密码表案例的启示：外部知识的”不完整”比”不存在”更具破坏性。当知识库提供的是经过截断、失真或片面化的信息时，智能体不仅无法从中获益，反而会被锁定在错误的搜索空间中。在构建 AutoPT 外部知识组件时，必须对知识的完整性、准确性和时效性进行严格校验。

5.2 基座模型能力不等于渗透测试表现

消融实验从”知识”维度揭示了框架设计的深层问题，而基座模型的选择则从”智能”维度进一步拓展了分析边界。一个自然的假设是：在通用 Benchmark 上表现更优的大模型，在 AutoPT 任务中也应当表现更好。然而，实验结果明确否定了这一假设。

5.2.1 GPT-5.2：探索薄弱与道德约束

GPT-5.2 在补充测试中展现出两个显著特征。首先是速度优势：其响应延迟明显低于其他模型，单位时间内可以完成更多的工具调用和推理轮次。然而，这一速度优势并未转化为更高的夺旗率。

根本原因在于探索能力的薄弱。GPT-5.2 在面对复杂靶场时，容易出现”过早终止”现象——即在尝试了少数几种攻击路径未果后，便迅速放弃并输出”无法利用”的结论。这种浅尝辄止的探索模式，在需要多步骤、多维度尝试的渗透测试场景中极为不利。日志中多次出现 GPT-5.2 在距离成功仅一步之遥时停止尝试的情况，例如已经识别出注入点但未尝试绕过过滤规则，或已经发现可疑接口但未进行进一步参数 fuzzing。

此外，GPT-5.2 表现出过强的道德约束和安全防护倾向。在渗透测试这一合法授权的安全评估场景中，模型频繁拒绝执行某些”看起来危险”的操作（如发送带有恶意 Payload 的 HTTP 请求），

即使这些操作在实验环境中完全合法且必要。这种过度保守的安全对齐，实质上构成了对任务执行的隐性约束。

5.2.2 Gemini Pro 3.1: 输出发散

Gemini Pro 3.1 的表现呈现出明显的“难度分化”特征。在简单难度的靶场上，该模型能够轻松识别漏洞并完成利用，其输出简洁、目标明确，执行效率较高。然而，当任务难度提升到中等或困难级别时，模型的输出开始出现严重的发散现象。

所谓“发散”，是指模型的推理链条不再聚焦于当前的攻击面，而是同时展开多条平行线索，且各线索之间缺乏优先级排序。例如，在同时存在 SQL 注入和文件上传两个潜在入口的场景中，Gemini Pro 3.1 可能在两条路径之间反复横跳：刚构造好一个注入 Payload，又突然转向尝试上传 WebShell，随后再次回到注入但更换了完全不同的技术路线。这种“布朗运动”式的探索模式，导致模型难以在任一方向上积累足够的深度，最终无法在有限的轮次内突破任何单一漏洞面。

GPT-5.2 与 Gemini Pro 3.1 的对比

- **GPT-5.2**: 速度快，但探索浅、终止早、道德约束过强；
- **Gemini Pro 3.1**: 简单任务高效，但复杂任务输出发散，难以聚焦突破口。

5.2.3 OS4.6: 最强但最贵

OS4.6 (Claude 4.6 系列) 在 AutoPT 任务中表现最强，无论是简单、中等还是困难难度的靶场，其夺旗率均显著高于其他模型。该模型展现出三个关键优势：

深度探索能力：OS4.6 不会轻易放弃，即使多次尝试失败，仍能在已有信息的基础上调整策略并继续探索。日志显示，OS4.6 在困难靶场上的平均尝试轮次明显高于 GPT-5.2 和 Gemini Pro 3.1。

输出聚焦性：与 Gemini Pro 3.1 的发散不同，OS4.6 的输出始终保持高度聚焦。即使在复杂场景中，模型也能识别出当前最有希望的攻击方向，并持续深入，避免无效的平行探索。

工具调用精准度：OS4.6 对工具参数的理解和构造更为准确，减少了因参数错误导致的执行失败和上下文浪费。

然而，OS4.6 的最强表现伴随着最高的成本。其 Token 单价显著高于 DS3.2 Chat，在消耗 100 亿 + Token 的实验规模下，成本差异被放大到不可忽视的程度。这一“性能-成本”权衡，是实际部署 AutoPT 框架时必须考虑的现实约束。

5.3 框架与模型的协同效应

基座模型的差异不仅体现在整体夺旗率上，更深刻地影响了框架内部的具体行为模式。一个极具说服力的案例来自 Trouble Competition 框架的子智能体调用。

如第 2 章所述，Trouble Competition 虽然名义上配备了子智能体 CYBERSPRK，但在主实验 (使用 DS3.2 Chat) 的全部 44 份日志中从未被调用过一次，因此被归类为“事实上的单智能体”。在补充测试中，这一模式在 GPT-5.2 和 Gemini Pro 3.1 上得到了延续：两个模型同样从未触发过子智能体调用。

唯一的例外发生在 OS4.6 上。在靶场 18 的测试中，OS4.6 调用了一次子智能体，并成功利用 XSS (Cross-Site Scripting, 跨站脚本攻击) 漏洞完成了夺旗。这是全部实验数据中唯一一次子智能体被成功调用的记录。

同

一框架、不同模型、截然不同的行为模式：Trouble Competition 的子智能体在 GPT-5.2 和 Gemini Pro 3.1 中从未被调用，仅在 OS4.6 中调用过一次并完成 XSS 利用。这说明框架的”设计能力”与模型的”执行能力”之间存在深度耦合——某些架构特性只有在特定模型的能力支撑下才能被激活。

这一案例深刻揭示了 AutoPT 框架设计的基本原则：框架不能脱离模型而独立存在。开发者在设计多智能体协作、工具编排、记忆管理等机制时，必须充分理解目标基座模型的特性——包括其探索倾向、输出风格、安全对齐强度以及工具调用习惯。一个为 OS4.6 优化的复杂多智能体架构，可能在 GPT-5.2 上完全失效；反之，一个适配 GPT-5.2 快速响应节奏的轻量级框架，又可能无法发挥 OS4.6 的深度推理优势。

框架-模型协同设计原则

1. **模型特性先行**：在选择框架架构前，先明确目标基座模型的核心能力边界和行为偏好；
2. **架构弹性设计**：框架应提供可配置的架构参数（如子智能体触发阈值、工具调用策略），以适应不同模型的特性；
3. **联合调优**：框架的超参数（如记忆压缩阈值、RAG 触发条件）应与具体模型联合调优，而非使用通用默认值。

5.4 本章小结

本章通过外部知识消融实验和基座模型对比分析，得出以下核心结论：

1. **传统 RAG 在渗透测试中大概率失效**。消融实验表明，删除外部知识组件后大部分框架分数不降反升。RAG 失效的三重根因——触发率低、内容不匹配、先验误导——共同构成了传统知识增强范式在 AutoPT 场景下的系统性障碍。唯一有效的例外是 CVE-POC 精确匹配，但其适用面过于狭窄，不具备可扩展性。
2. **外部知识的”质量”比”数量”更重要**。Closer 密码表案例表明，不完整或失真外部知识的破坏力远超没有外部知识。构建 AutoPT 知识库时，必须对知识的完整性、准确性和环境适配性进行严格校验，避免”半桶水”式的知识注入。
3. **通用 Benchmark 领先不等于 AutoPT 最优**。GPT-5.2 的探索薄弱与道德约束、Gemini Pro 3.1 的复杂任务发散，均说明通用能力评估无法直接迁移到渗透测试这一高度专业化的任务场景。OS4.6 虽最强但最贵，实际部署需在性能与成本之间权衡。
4. **框架设计必须与基座模型深度绑定**。Trouble Competition 子智能体在不同模型下的调用差异证明，同一架构在不同模型上可能呈现完全不同的行为模式。AutoPT 框架的设计者必须将模型特性作为架构决策的核心输入，而非事后的适配对象。

下一章将从”知识”和”模型”转向更底层的”工具”和”执行”层面，通过对工具调用规模、原子工具地位以及两个具体靶场（靶场 22 和靶场 026）的深度分析，进一步揭示 AutoPT 框架在执行层的共性问题与实战洞察。

6 工具调用、靶场分析与实战洞察

第 5 章从“知识”和“模型”两个维度揭示了 AutoPT 框架的上层设计陷阱。然而，无论知识库如何配置、基座模型如何选择，框架最终都必须通过底层工具的调用来与目标环境交互。工具是智能体作用于现实世界的“手脚”，工具调用的质量直接决定了渗透测试的执行效果。本章首先分析工具规模与表现之间的非单调关系，论证原子工具的核心地位，随后剖析执行层的三大共性问题，最后深入两个具体靶场——靶场 22（多漏洞利用）和靶场 026（已知 CVE 漏洞利用）——从实战日志中提炼关于推理能力下限与记忆机制上限的关键洞察。

6.1 工具数量不等于表现

一个直观的工程假设是：为智能体配备越多的工具，其能力边界就越广，表现也应当越好。研究团队在对 15 个框架的工具配置进行统计后，发现这一假设在 AutoPT 场景中并不成立。

工具被调用的规模与框架的夺旗率之间不存在单调相关性。某些配备了数十种专业安全工具的框架，其表现反而落后于仅依赖少数基础工具的框架。深入日志分析后发现，“拥有工具”与“懂得使用工具”之间存在巨大的鸿沟。当工具数量过多时，模型面临的选择空间急剧膨胀，反而增加了决策难度——模型可能在多个功能相近的工具之间犹豫不决，或者选择了不适用于当前场景的工具，导致执行失败和上下文浪费。

工

具数量不等于表现：给智能体配备一堆工具，并不等同于它懂得如何使用这些工具。盲目堆砌工具不仅无法提升夺旗率，反而可能因决策空间膨胀和上下文过载而降低整体表现。

这一现象与第 5 章中 RAG 失效的“触发率低”问题形成了有趣的平行关系：无论是知识库还是工具库，“配置存在”与“有效利用”之间都隔着一层模型理解和调度的能力屏障。框架设计者不能简单地将开源工具或商业工具堆砌到智能体的工具箱中，而必须考虑模型对工具语义的理解程度、工具之间的协调机制，以及工具描述信息对上下文的占用。

6.2 原子工具的核心地位

在工具调用的光谱中，研究团队识别出了一类特殊的工具——原子工具（Atomic Tools）。原子工具是指最基础、不可再分的工具单元，典型代表包括 `curl`（HTTP 请求）、`Python`（脚本执行）、`nc`（网络连接）等。与 Nmap、Metasploit、Burp Suite 等专业安全工具相比，原子工具功能单一、接口简单，但具备最高的灵活性和通用性。

实验日志显示，原子工具才是支撑所有渗透测试任务的底层贡献组件。无论框架配备了多少高级工具，当面临一个非标准或经过修改的漏洞场景时，模型最终都会退化为依赖原子工具进行手动编排——用 `curl` 构造自定义请求、用 `Python` 编写临时脚本来解析响应或生成 Payload。这种“退化”并非偶然，而是反映了原子工具在渗透测试中的不可替代性：专业工具的高级功能往往建立在特定假设之上（如标准漏洞模式、已知服务指纹），一旦目标环境偏离这些假设，专业工具便失效，而原子工具因其底层性仍能发挥作用。

原子工具 vs. 复合工具

- **原子工具**：功能单一、接口简单、灵活性高（如 `curl`、`Python`、`nc`）；
- **复合/专业工具**：功能丰富、接口复杂、针对性强（如 `Nmap`、`Metasploit`、`SQLMap`）；
- **关键差异**：原子工具在非标场景下仍能工作，而专业工具在假设失效时完全失效。

然而，这种对原子工具的依赖也带来了显著的局限性。当框架缺乏专业安全工具时，它退化为依赖原子工具进行手动编排以实现功能替代。这种机制虽然能够完成基础任务，却很难复现专业工具的高级执行能力。例如，用 `curl` 和 `Python` 手动实现 SQL 注入检测是可行的，但要复现 `SQLMap` 的自动数据库枚举、WAF 绕过、多线程并发等高级特性，则需要大量的额外代码和复杂的逻辑编排，远超当前 LLM 在单次任务中的代码生成能力。

6.3 执行层共性问题

除了工具本身的选型和规模问题，研究团队在日志中还识别出了执行层的三大共性缺陷，这些缺陷横跨多个框架，构成了 AutoPT 工程实现中的普遍性挑战。

第一，交互式工具引发流程阻塞。许多安全工具（如 `ssh`、交互式 Shell、需要多轮确认的安装脚本）在执行过程中会等待用户输入。AutoPT 框架若未对这些交互式工具进行特殊处理，智能体在调用后会陷入无限等待，导致整个执行流程阻塞。日志中多次出现模型调用某工具后，后续数十轮无任何进展的情况，最终因超时而被判定为失败。

第二，工具信息膨胀撑爆上下文。部分工具的输出体积极为庞大（如 `Nmap` 的全端口扫描、目录爆破工具的递归输出）。当这些输出未经截断或摘要直接回传给模型时，会迅速占据大量上下文窗口，挤压推理和决策所需的空间。第 3 章中提到的 CTF Server 框架因 FAZ 输出控制不当而撑爆上下文，正是这一问题的典型案例。

第三，无约束命令带来安全风险。AutoPT 框架通常以高权限运行，智能体执行的命令直接作用于目标系统。若框架未对命令进行白名单校验、沙箱隔离或行为审计，模型可能执行破坏性操作（如 `rm -rf /`、系统服务重启），不仅对靶场环境造成不可逆损害，更在真实部署中构成了严重的安全隐患。

执

行层三大风险：交互式工具阻塞、工具输出膨胀撑爆上下文、无约束命令带来安全风险。这三类问题并非某个框架的独有缺陷，而是 AutoPT 工程实现中的普遍性挑战，需要在 Harness 层进行系统性防护。

6.4 靶场 22：多漏洞利用分析

靶场 22 是一个典型的多漏洞利用场景，要求攻击者依次或组合利用多个漏洞才能最终获取 Flag。这类场景对智能体的综合能力提出了极高要求：不仅需要识别单个漏洞，更需要理解漏洞之间的依赖关系，规划多步骤的攻击链。

然而，日志分析揭示了一个令人警醒的事实：超过 70% 的日志显示，模型根本没有进行多漏洞利用。在这些失败的执行轨迹中，智能体往往只识别并利用了其中一个漏洞，随后便停止了探索，未能发现后续漏洞的存在。这导致大量“半途而废”的失败案例——模型已经迈出了第一步，却因缺乏持续探索的动机或机制，在距离成功仅几步之遥的地方停下。

研究团队进一步对比了成功与失败框架的特征，发现了一个关键差异：能够稳定识别攻击面并完成多漏洞利用的框架，全部具备显式的关键信息存储机制。这些框架在发现第一个漏洞后，会主动将关键信息（如发现的服务版本、可疑接口、潜在注入点）写入记忆系统，并在后续轮次中显式检索和利用这些信息。这种“发现-记录-复用”的闭环，使得模型能够在多轮交互中积累对目标环境的认知深度，从而逐步构建完整的攻击链。

靶

场 22 的核心发现：70%+ 的日志未进行多漏洞利用。成功框架的共同特征是具备显式关键信息存储机制。记忆不是“锦上添花”，而是多步骤攻击链得以延续的必要基础设施。

为了进一步验证这一结论，研究团队进行了一个精巧的对照实验：将三个单智能体框架的基座模型由 DS3.2 Chat 替换为 OS4.6，其他配置保持不变。结果三个框架全部能够稳定完成多漏洞利用。这一结果揭示了 AutoPT 中两个关键因素的互补关系：

- **LLM 推理能力决定多漏洞利用组合的下限。**更强的基座模型（如 OS4.6）具备更优的漏洞识别、攻击链规划和异常处理能力，即使在没有显式记忆辅助的情况下，也能通过更强的上下文理解和推理能力完成多步骤利用。
- **显式记忆机制提升多漏洞利用的上限。**在相同模型能力下，具备良好记忆管理的框架能够更稳定、更高效地完成多漏洞利用，减少因信息遗忘导致的重复探索和方向偏离。

推理能力下限与记忆机制上限

$$\text{多漏洞利用成功率} = f(\underbrace{\text{基座模型推理能力}}_{\text{决定下限}}, \underbrace{\text{显式记忆机制质量}}_{\text{提升上限}})$$

基座模型能力决定了“能不能做”，记忆机制决定了“做得稳不稳”。两者缺一不可，且无法相互完全替代。

6.5 靶场 026：已知 CVE 漏洞利用

靶场 026 是一个关于已知 CVE 漏洞利用的场景。与靶场 22 的多漏洞组合不同，靶场 026 的核心挑战在于：模型需要正确识别目标系统存在的 CVE 编号，并构造有效的利用 Payload。这一场景直接检验了 AutoPT 框架对公开漏洞知识的获取、关联和转化能力。

实验数据显示：16.7% 的日志未能关联到正确的 CVE 编号；在此基础上，56.7% 的日志虽然关联到了正确的 CVE，却未能构造出有效的 Payload。这两个数字叠加后，意味着绝大多数框架在已知 CVE 场景下的表现远未达到理想水平。

```
1 # 模式一：CVE 关联错误（16.7% 日志）
2 # 模型将目标漏洞误判为其他 CVE
3 identified_cve = "CVE-2023-XXXX" # 错误
4 actual_cve = "CVE-2024-YYYY" # 正确
5
6 # 模式二：CVE 正确但 Payload 无效（56.7% 日志）
7 # 模型知道漏洞类型，但构造的 Payload 未适配目标环境
8 payload = "generic_payload_for_CVE" # 通用 Payload
9 # 目标环境经过修改，需要调整特定参数或绕过过滤
```

Listing 3: 靶场 026 CVE 利用失败模式示意

在所有被测框架中，唯一能够稳定攻破靶场 026 的是 CTF Server——去年比赛的第二名。该框架针对特定 CVE 做了专门的 POC，这使其在 Payload 构造环节具有显著优势。然而，研究团队在消融实验中将该 POC 删除后，CTF Server 也无法稳定攻破靶场 026。这一结果说明，CTF Server 的成功并非源于其框架架构的优越性，而是高度依赖于预置的精确外部知识——一旦移除这一“拐杖”，其表现便迅速退化到与其他框架相近的水平。

C

TF Server 的“成功”具有高度条件性：其稳定攻破靶场 026 的能力完全依赖于针对特定 CVE 的预置 POC。删除 POC 后同样无法稳定利用。这意味着当前 AutoPT 框架对已知 CVE 的利用能力，本质上仍停留在“硬编码特例”阶段，而非具备通用的漏洞知识转化能力。

安全世界每天都在涌现新的漏洞，而大模型内化的参数知识存在时效性天花板——训练数据截止后发布的 CVE 不可能被模型直接掌握。因此，构建有效的外部知识库驱动范式，是实现公开漏洞持续利用的唯一可行路径。这一结论与第 5 章中“传统 RAG 大概率无效”的发现形成了辩证关系：外部知识是必要的，但传统 RAG 的实现方式是错误的。AutoPT 领域亟需一种新型的知识驱动架构，能够在保证知识精确性、时效性和环境适配性的前提下，将公开漏洞信息有效转化为可执行的攻击能力。

6.6 推理能力下限与记忆机制上限

综合靶场 22 和靶场 026 的分析，可以提炼出 AutoPT 框架设计的两个核心维度：推理能力与记忆机制。

推理能力是基座模型的固有属性，决定了智能体在面对未知场景时的“下限”——即在最不利条件下（无外部知识、无历史记忆、无预置 POC）仍能完成的基础任务范围。OS4.6 之所以在替换实验中让三个单智能体框架全部稳定多漏洞利用，正是因其更强的推理能力提升了这一下限。然而，推理能力并非无限，即使是最强的模型，在超长任务链中也难免出现信息遗忘和方向偏离。

记忆机制是框架的工程实现，决定了智能体在复杂任务中的“上限”——即在有利条件下（有显式信息存储、有结构化检索）能够达到的最佳表现。靶场 22 中成功框架的共性特征，正是记忆机制对上限的提升作用。但记忆机制本身无法替代推理：如果基座模型无法理解存储的信息、无法基于检索结果进行有效推理，再完善的记忆系统也只是“垃圾进、垃圾出”。

推

理能力决定下限，记忆机制提升上限。AutoPT 框架的优化必须在这两个维度上协同发力：既要选择推理能力足够强的基座模型，又要构建能够有效存取和复用关键信息的记忆系统。任何单一维度的极端优化都无法突破 AutoPT 的整体性能瓶颈。

6.7 本章小结

本章从工具调用和靶场实战两个层面，揭示了 AutoPT 框架在执行层的深层问题与关键洞察：

1. **工具规模与表现无单调相关性。**盲目堆砌工具不仅无法提升夺旗率，反而可能因决策空间膨胀而降低表现。原子工具（curl、Python 等）因其底层性和灵活性，是所有渗透测试任务的最

终执行基石；但过度依赖原子工具也会导致框架退化为手动编排，难以复现专业工具的高级能力。

2. **执行层存在三大共性风险**：交互式工具引发的流程阻塞、工具信息膨胀对上下文的撑爆效应，以及无约束命令带来的安全隐患。这些问题需要在 Harness 层进行系统性防护，而非依赖模型自身的判断。
3. **多漏洞利用的成功依赖于显式记忆机制**。靶场 22 的分析表明，70%+ 的失败源于模型未能进行多漏洞组合利用。成功框架的共同特征是具备显式关键信息存储，而基座模型替换实验 (DS3.2 → OS4.6) 进一步验证了“推理能力决定下限、记忆机制提升上限”的核心规律。
4. **已知 CVE 的利用仍停留在“硬编码”阶段**。靶场 026 的数据显示，绝大多数框架无法正确关联 CVE 或构造有效 Payload。CTF Server 的唯一成功高度依赖预置 POC，删除后同样失效。这说明构建有效的外部知识库驱动范式，是实现公开漏洞持续利用的必经之路——但传统 RAG 并非答案。

工具是手脚，记忆是大脑，模型是灵魂。三者缺一不可，且必须深度协同。第 7 章将在前六章全部发现的基础上，综合提出 AutoPT 领域的未来演进方向，从记忆架构重建到领域适配 RAG，从安全隔离到自动化日志审计，为这一复杂工程领域的持续发展勾勒出系统性的路线图。

7 总结与延伸

本章将前六章的实验发现与根因分析综合为系统性的核心论断，并在此基础上提出 AutoPT 领域未来演进的七大方向，最终回归到一个根本问题：自动渗透测试究竟是一项什么性质的任务。

7.1 核心发现回顾

经过对 Trouble 靶场上 15 个框架、1500+ 份日志的人工审查，本研究在六维度体系下提炼出以下九大核心发现：

九大核心发现

1. **单智能体具备强大竞争力。**标准 ReAct 循环下的单智能体在前六名中占据一半席位，其上下文完整、决策-执行-反馈链路短、零通信开销的特性，使其天然适配 CTF 场景。
2. **多智能体上下文拆分与 Token 高消耗。**多智能体的复杂性并未带来性能提升，反而因角色切换、信息传递和冗余通信引入大量 Token 开销与 Bug。
3. **记忆管理占据核心地位。**十个框架存在记忆系统形同虚设的问题，包括 CHY 的 ADD memory 工具未注册、Teddy City 笔记工具调用率极低、HPT Index 压缩阈值过高等。在 AutoPT 中，**记忆比规划更重要**。
4. **传统 RAG 的负面作用。**消融实验表明，删除外部知识组件后大部分框架分数不降反升；传统 RAG 检索内容与目标环境不匹配，易误导智能体偏离实际漏洞面。
5. **盲目堆砌工具的局限性。**工具数量与框架表现无单调相关性，配备大量工具不等于懂得如何使用。
6. **原子工具补偿机制的有效性。**curl、Python 等原子工具是支撑所有任务的底层组件，在困难任务中通过手动编排实现功能替代，虽难以复现专业工具的高级能力，但构成有效的执行兜底。
7. **通用 AI 编程智能体的惊艳得分能力。**CodeX、Cloud Code 等通用编程智能体展现出远超预期的渗透测试得分，提示 Harness 工程完备性的关键价值。
8. **模型适配需求。**通用 Benchmark 领先不等于 AutoPT 最优；同一框架内不同基座模型 (DS3.2 Chat、OS4.6、GPT-5.2、Gemini Pro 3.1) 的任务规划与工具调用行为差异显著。
9. **动态维护 CVE 智库的必要性。**靶场 026 数据显示，16.7% 日志未关联正确 CVE，56.7% 未构造有效 Payload；唯有构建有效的外部知识库驱动范式，才能实现对公开漏洞的持续利用。

上述发现并非孤立的技术观察，而是指向一个更深层的问题：AutoPT 框架的设计者在何处做出了错误的工程假设。单智能体的成功暗示了”复杂性不等于能力”；记忆失效的普遍性暗示了”有工具不等于有机制”；RAG 的负面作用暗示了”有知识不等于有帮助”。这些发现共同构成了一幅 AutoPT 工程现状的全景图——许多框架在纸面架构上看似完备，但在实际执行中因工程细节失守而崩溃。

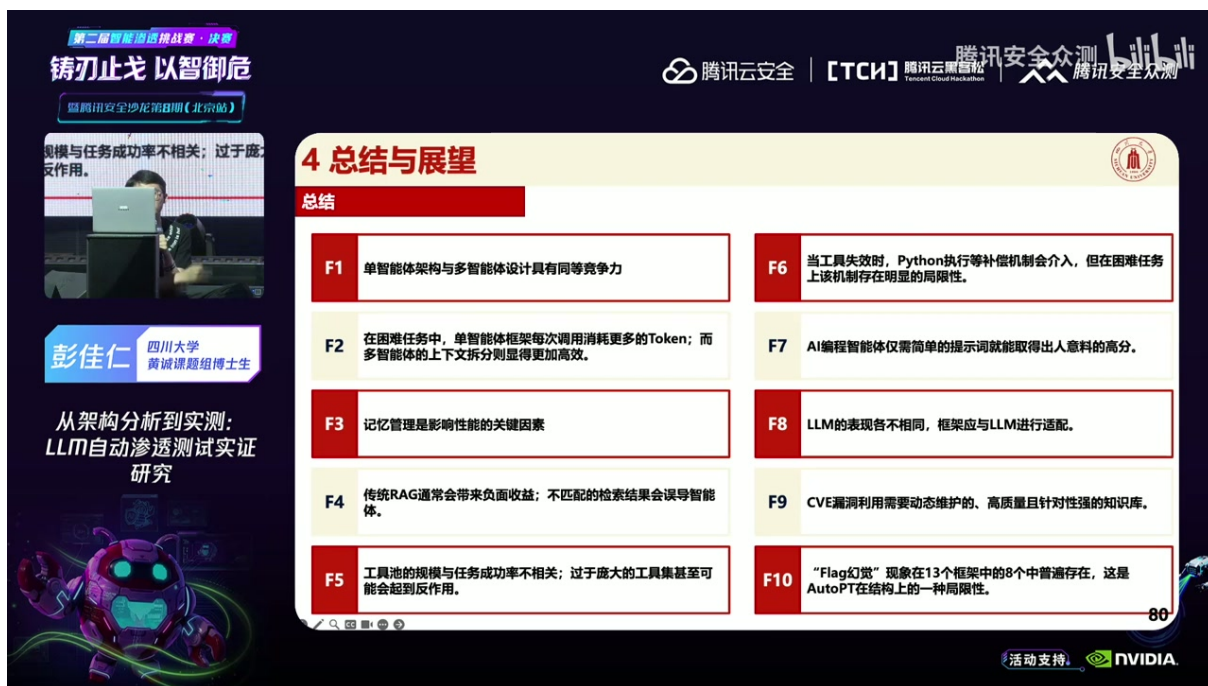


图 9: 九大核心发现的逻辑关联：从架构假设到工程现实的落差⁹

7.2 未来演进方向

基于前述发现，演讲者提出了 AutoPT 领域未来需要重点攻克的七大方向。这些方向并非泛泛而谈的研究建议，而是与前文实验数据一一对应的工程改进路径。

7.2.1 记忆管理与架构重建

记忆系统失效是本研究中发现最广泛的问题类型。未来的 AutoPT 框架必须建立**显式、合理的信息存取机制**，而非简单地将记忆功能作为可选工具注册进系统。

显式记忆机制的设计原则

- **写入触发**：明确什么信息值得被记忆（关键发现、攻击面、中间凭证），而非被动等待模型调用记忆工具。
- **读取策略**：在决策关键节点自动注入相关记忆，而非依赖模型主动检索。
- **压缩阈值**：避免过早压缩导致信息丢失（如 HPT Index 的 6400 Token 阈值问题）。
- **多智能体决策划分**：多智能体架构必须提供明确的角色边界和信息流转规则，避免子智能体从未被调用或意见相互矛盾的情况重演。

7.2.2 反思机制

高质量的反思机制依赖于准确的反馈信号，而准确的反馈信号又依赖于可靠的记忆系统。这是一个典型的**依赖链问题**：没有好的记忆，就没有好的反思；没有好的反思，就无法从失败中迭代改进。

⁹视频画面时间区间：00:12:10–00:12:40。

当前框架中，反思往往流于形式——模型在每一步后生成一段“自我批评”文本，但这段文本既不被结构化存储，也不影响后续决策。未来的反思机制需要：

1. 将反思结果写入结构化记忆，而非仅作为一次性的推理附言；
2. 建立反馈信号的验证机制，避免模型被自身的错误假设反复强化；
3. 在任务失败时触发根因回溯，而非简单终止执行。

7.2.3 工具使用指导

实验明确表明，盲目堆砌工具不等于提升能力。未来的框架设计需要在两个层面改进：

工具设计的双重陷阱

1. **使用指导缺失**：仅向智能体提供工具列表和参数签名是不够的，必须告知工具的使用场景、调用时机和输出解读方式。
2. **健壮性不足**：交互式工具（如需要用户输入的 shell 命令）常引发流程阻塞；超长回显（如大规模扫描输出）易撑爆上下文。执行层必须具备超时处理、输出截断和流式处理机制。

特别值得注意的是，Python 等原子工具在困难任务中的补偿作用提示了一个设计权衡：与其追求工具数量，不如确保核心原子工具的调用健壮性和输出可控性。

7.2.4 领域适配 RAG

传统 RAG 范式在渗透测试中大概率无效，这是本研究中最具颠覆性的发现之一。消融实验表明，除非知识库恰好包含针对特定 CVE 的 POC，否则 RAG 检索往往提供错误的先验知识，将智能体引向错误方向。

未来的领域适配 RAG 需要重新设计三个核心环节：

- **索引方式**：从通用文本索引转向漏洞-centric 的结构化索引（CVE 编号、受影响版本、攻击向量、Payload 模板）。
- **检索策略**：结合当前目标环境的指纹信息（开放端口、服务版本、Web 技术栈）进行定向检索，而非基于自然语言查询的模糊匹配。
- **来源验证**：对检索到的知识进行可信度评估，优先采用经过验证的 POC，而非通用描述性文本。

7.2.5 安全隔离与沙箱

渗透测试智能体天然掌握高系统权限——它可以执行任意命令、访问敏感文件、发起网络攻击。这意味着 AutoPT 框架本身就是一个**高价值攻击面**。

安全风险的双重维度

1. **目标环境隔离**: 智能体对靶场的操作必须在沙箱中进行, 防止逃逸到宿主机或生产网络。
2. **工具执行约束**: 无约束的命令执行 (如任意 shell 命令) 可能带来不可逆的破坏, 需要建立命令白名单、影响范围评估和回滚机制。

当前开源框架在安全隔离方面的设计普遍薄弱, 这不仅是技术债, 更是阻碍 AutoPT 从实验室走向实际部署的关键瓶颈。

7.2.6 模型与框架协同

实验中最具启示性的发现之一是: 同一框架在不同基座模型下的表现差异巨大。Trouble Competition 的子智能体在 GPT-5.2 和 Gemini Pro 3.1 中从未被调用, 仅在 OS4.6 中调用过一次完成 XSS 漏洞利用。

这揭示了一个根本原则: **AutoPT 框架设计必须与底层大模型的特征深度绑定**。不同模型在以下维度存在显著差异:

- 工具调用偏好 (是否倾向于使用特定工具)
- 探索-利用权衡 (是否愿意持续尝试不同攻击向量)
- 安全对齐强度 (是否因道德约束而拒绝执行某些操作)
- 上下文利用效率 (能否在长上下文中保持关键信息不丢失)

框架设计者必须了解所选用模型的”癖好和偏好”, 并据此调整提示工程、工具注册和记忆策略。

7.2.7 自动化日志审计

本研究之所以能够发现上述问题, 核心方法论在于对 1500+ 份日志的人工审查。然而, 这种依赖人力的大规模审计不可持续。

自动化日志审计的价值

演讲者特别呼吁社区建立自动化日志审计机制。通过结构化日志分析, 框架开发者可以系统性地发现以下问题:

- 工具注册但从未被调用的”僵尸工具”
- 记忆工具的低频使用或错误使用模式
- 多智能体间的通信失败和角色越界
- 上下文膨胀的时间点和触发源
- 重复性失败的任务模式

自动化日志审计不仅是调试手段, 更应成为 AutoPT 框架持续迭代的闭环基础设施。

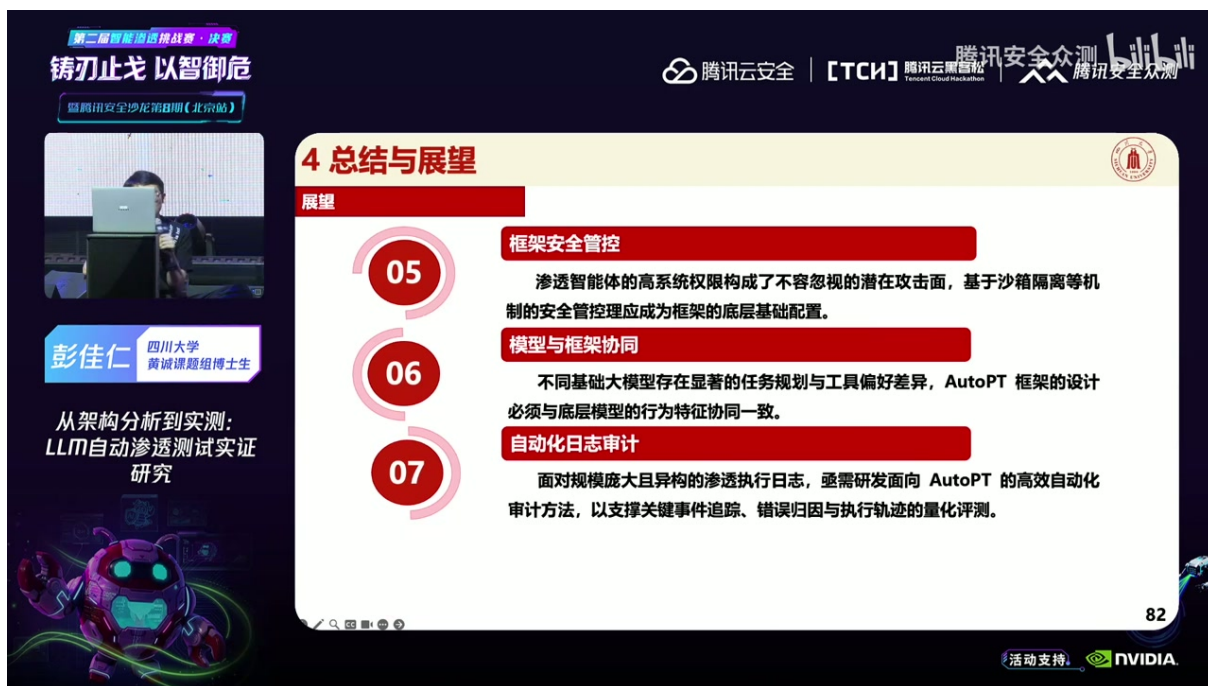


图 10: AutoPT 未来七大演进方向及其与前文发现的对应关系¹⁰

7.3 最终结论：AutoPT 的工程复杂性

演讲者在结语中给出了一个强有力的论断：

AutoPT 不是一项简单的任务，而是一个极其复杂、极其典型的 AI 工程。

这一论断的深刻性在于，它将 AutoPT 从“用 LLM 做渗透测试”的技术噱头，重新定位为对智能体工程能力的全面压力测试。AutoPT 同时要求以下能力在同一系统中协同工作：

AutoPT 的工程能力矩阵

- **规划范式**：在高度不确定的攻击面中制定和迭代行动计划；
- **记忆管理**：在长时间、多步骤的任务中保持关键信息不丢失；
- **高频工具调用**：不仅是标准化 API 调用，更包括交互式命令、不稳定输出和工具崩溃的处理；
- **安全隔离**：在高权限执行环境下保障系统和数据安全；
- **知识库更新**：动态吸收每日涌现的新漏洞知识；
- **Benchmark 迭代**：持续设计和维护有区分度的评估标准。

演讲者进一步指出：**编程智能体（如 CodeX、Cloud Code）做得好，不是因为它简单，而是因为它做得足够复杂，Harness 做得好才能成功。**这里的 Harness 指的是智能体的执行框架——负责调度智能体、执行命令、收集反馈、管理上下文的完整工程系统。

¹⁰ 视频画面时间区间：00:12:50–00:13:40。

这一观点对于 AutoPT 社区具有直接的警示意义。当前许多开源框架将主要精力投入到架构设计（多智能体角色分配、规划算法）和工具集成（支持更多安全工具）上，却在 Harness 的健壮性上投入不足。交互式命令阻塞、上下文膨胀、记忆失效、日志缺失——这些“不性感”的工程问题，恰恰是决定框架能否从演示走向实用的分水岭。

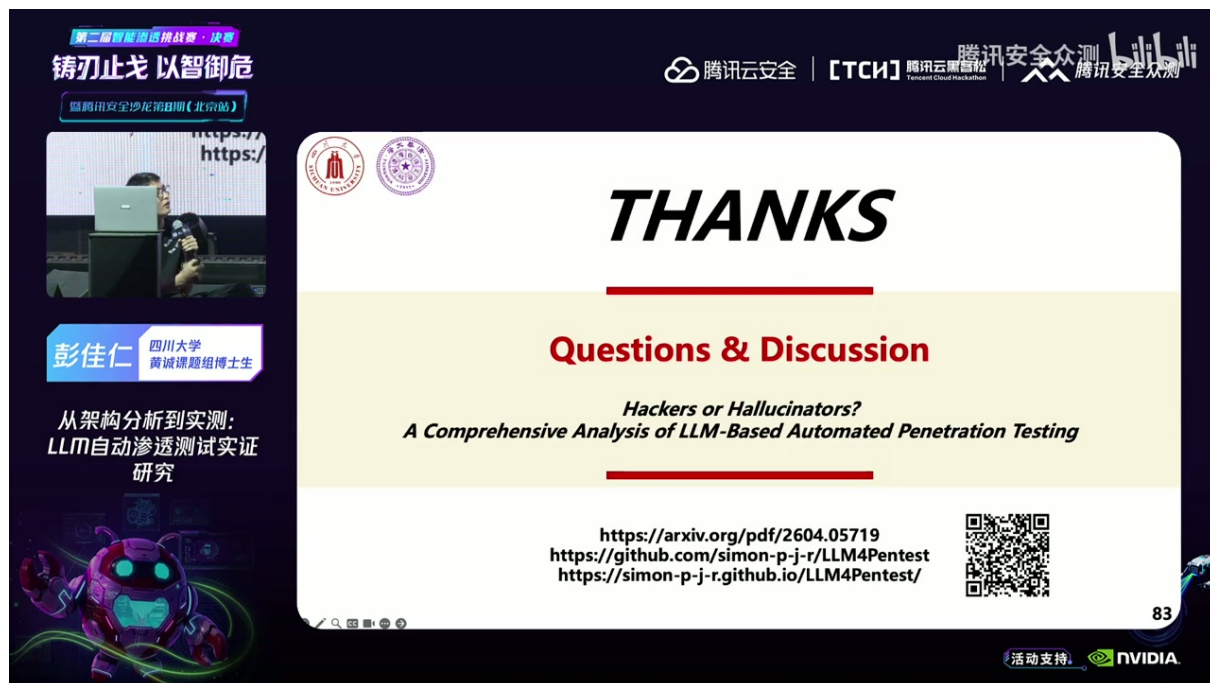


图 11: 演讲者最终结论：AutoPT 作为 AI 工程的复杂性本质¹¹

从更宏观的视角看，AutoPT 可以被视为智能体工程的**压力测试**（Stress Test）。它将规划、记忆、工具、安全、知识五个维度的挑战压缩到一个系统中，并要求它们在对抗性环境下协同工作。在这个意义上，AutoPT 的研究价值不仅在于提升渗透测试的自动化水平，更在于它为通用智能体工程提供了一个极端但真实的检验场——一个在失败代价明确、成功标准清晰、环境动态变化的场景中，验证智能体设计假设的试金石。

7.4 本章小结

本章系统回顾了本研究的九大核心发现，从单智能体的竞争力、记忆管理的核心地位、传统 RAG 的负面作用，到模型适配需求和 CVE 智库维护的必要性。在此基础上，提出了 AutoPT 领域未来演进的七大方向：记忆管理与架构重建、高质量反思机制、工具使用指导、领域适配 RAG、框架安全与沙箱隔离、模型与框架深度协同、自动化日志审计。

最终，本章将 AutoPT 重新定位为**智能体工程的压力测试**——一个将规划、记忆、工具、安全、知识所有挑战压缩到单一系统中的复杂 AI 工程。编程智能体的成功经验表明，Harness 的完备性而非架构的华丽程度，才是决定智能体系统成败的关键。对于 AutoPT 社区而言，这意味着需要将工程重心从“增加更多功能”转向“确保现有功能可靠运行”，从“设计更复杂的架构”转向“打磨更健壮的执行框架”。

¹¹ 视频画面时间区间：00:14:15–00:15:10。